JAVA ASSESSMENT PLATFORM

Adaptive Quiz & Exam Engine

Dynamic question difficulty scaling, type-safe generic collections, and secure interactive examination workflows in Java.

System Abstract

The **Adaptive Quiz & Exam Engine** is a robust Java application architected to dynamically deliver questions, track user responses in real-time via the `Scanner` class, and adjust assessment difficulty based on performance metrics.

It synthesizes core Object-Oriented Programming (OOP) paradigms, generic collections, lambda expressions, and file I/O operations into a modular, production-ready testing platform.

Core Platform Advantages

Adaptive Scaling

Modulates subsequent question difficulty based on prior user accuracy.

Type Safety

Enforces compile-time checking through Java Generics and strong OOP encapsulation.

Persistent Logging

Loads question banks and exports exam report transcripts via BufferedReader/PrintWriter.

OOP Architecture & Modifiers

Encapsulation & Getters/Setters

Private state fields protected with public accessor and mutator methods for secure data management.

Abstraction & Inheritance

Base abstract class `Question` implements core fields while subclasses inherit behavior using `super`.

Polymorphism & Overriding

Runtime method overriding allows specialized evaluation rules for MCQ, True/False, and Coding questions.

Advanced Java Modules (5 & 6)

Strings & File I/O

Utilizes string manipulation (`trim()`, `equalsIgnoreCase()`) alongside `BufferedReader` and `PrintWriter`.

Generics & Boxing

Type-safe collections (`List`) with automatic autoboxing and unboxing of score wrappers.

Iterators & Lambdas

Iterating collections using explicit `Iterator` loops and functional lambda filtering for analytics.

Engine Code Implementation

```
import java.util.*;
abstract class Question {
    private String text; private int difficulty;
    public Question(String text, int difficulty) { this.text = text; this.difficulty = difficulty; }
    public String getText() { return text; }
    public int getDifficulty() { return difficulty; }
    public abstract boolean evaluate(String answer);
}
class MCQQuestion extends Question {
    private String correctAnswer;
    public MCQQuestion(String text, int difficulty, String correctAnswer) {
        super(text, difficulty); this.correctAnswer = correctAnswer;
    }
    @Override public boolean evaluate(String answer) {
        return this.correctAnswer.equalsIgnoreCase(answer.trim());
    }
}
public class QuizEngine {
    public static void main(String[] args) {
        List bank = new ArrayList<>();
        bank.add(new MCQQuestion("Java is object-oriented? (y/n)", 1, "y"));
        Scanner scanner = new Scanner(System.in);
        Integer score = 0; // Autoboxing
        Iterator it = bank.iterator();
        while(it.hasNext()) {
            Question q = it.next();
            System.out.println(q.getText());
            String ans = scanner.nextLine(); // Capturing user input via Scanner
            if(q.evaluate(ans)) score += 10; // Unboxing
        }
        System.out.println("Final Calculated Score: " + score);
        scanner.close();
    }
}
```

Demonstrates Scanner class interactive input, encapsulation, abstract class, inheritance, super, overriding, iterators, and autoboxing.

Execution Output & File Persistence

Interactive Console Session

```
Java is object-oriented? (y/n)
> y
Final Calculated Score: 10
```

File I/O Stream Integration

Using `BufferedReader` to ingest structured questions from external storage and `PrintWriter` to record audit logs of student exam sessions.

🏁 SUMMARY & ROADMAP

Conclusion & Future Work

Conclusion

Successfully synthesized core object-oriented principles, generic collections, and interactive scanner I/O streams into a robust assessment tool.

Future Work

Integrating Item Response Theory (IRT) algorithms, multi-threaded exam timers, and persistent database storage via JDBC connections.